Jenkins Postman FastAPI

Fast API code

```python
load_dotenv()

app = FastAPI()

MONGO_URL = os.getenv("MONGO_URL", "mongodb://localhost:27017")
DB_NAME   = os.getenv("DB_NAME", "Products")
COLLECTION = "Products"


@app.on_event("startup")
async def startup():
    app.mongodb_client = AsyncIOMotorClient(MONGO_URL)
    app.db = app.mongodb_client[DB_NAME]

@app.on_event("shutdown")
async def shutdown():
    app.mongodb_client.close()


def item_to_dict(item: dict) -> dict:
    item["ProductID"] = item.pop("_id")
    return item

class ItemCreate(BaseModel):
    ProductID: int
    Name: str
    UnitPrice: float
    StockQuantity: int
    Description: Optional[str]

class ItemResponse(BaseModel):
    ProductID: int
    Name: str
    UnitPrice: float
    StockQuantity: int
    Description: Optional[str] = None

class ConvertResponse(BaseModel):
    ProductID: int
    Name: str
    price_usd: float
    price_eur: float
    exchange_rate: float
    rate_date: str


# /getSingleProduct - This allows you to pass a single ID number into the endpoint and return the details of the single product in JSO
@app.get("/getSingleProduct", response_model=ItemResponse)
async def get_single_product(id: int):
    item = await app.db[COLLECTION].find_one({"_id": id})
    if item is None:
        raise HTTPException(status_code=404, detail=f"Product with ID {id} not found")
    return item_to_dict(item)


# /getAll - This endpoint should return all inventory in JSON format from the database.
@app.get("/getAll", response_model=list[ItemResponse])
async def get_all():
    items = app.db[COLLECTION].find()
    results = [item_to_dict(item) async for item in items]
    if not results:
        raise HTTPException(status_code=404, detail="No products found in the database")
    return results

# /addNew - This endpoint should take in all 5 attributes of a new item and insert them into the database as a new record.
@app.post("/addNew", response_model=ItemResponse, status_code=status.HTTP_201_CREATED)
async def add_new(item: ItemCreate):
    existing = await app.db[COLLECTION].find_one({"_id": item.ProductID})
    if existing:
        raise HTTPException(status_code=409, detail=f"Product with ID {item.ProductID} already exists")
    item = item.model_dump()
    item["_id"] = item.pop("ProductID")
    await app.db[COLLECTION].insert_one(item)
    created = await app.db[COLLECTION].find_one({"_id": item["_id"]})
    if created is None:
        raise HTTPException(status_code=500, detail="Product was not created successfully")
    return item_to_dict(created)
```

```python
# /deleteOne - This endpoint should delete a product by the provided ID.
@app.delete("/deleteOne", status_code=status.HTTP_204_NO_CONTENT)
async def delete_one(id: int):
    result = await app.db[COLLECTION].delete_one({"_id": id})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail=f"Product with ID {id} not found")


# /startsWith - This should allow the user to pass a letter to the URL, such as s, and return all products that start with s.
@app.get("/startsWith", response_model=list[ItemResponse])
async def starts_with(letter: str):
    if len(letter) != 1 or not letter.isalpha():
        raise HTTPException(status_code=400, detail="Parameter must be a single alphabetic letter")
    items = app.db[COLLECTION].find(
        {"Name": {"$regex": f"^{letter}", "$options": "i"}}
    )
    results = [item_to_dict(item) async for item in items]
    if not results:
        raise HTTPException(status_code=404, detail=f"No products found starting with '{letter}'")
    return results


# /paginate - This URL should pass in a product ID to start from and a product ID to end from. The products should be returned in a ba
@app.get("/paginate", response_model=list[ItemResponse])
async def paginate(start_id: int, end_id: int):
    if start_id > end_id:
        raise HTTPException(status_code=400, detail="start_id must be less than or equal to end_id")
    items = (
        app.db[COLLECTION]
        .find({"_id": {"$gte": start_id, "$lte": end_id}})
        .sort("_id", 1)
        .limit(10)
    )
    results = [item_to_dict(item) async for item in items]
    if not results:
        raise HTTPException(status_code=404, detail=f"No products found between ID {start_id} and {end_id}")
    return results

# /convert - All of the prices are currently in dollars in the sample data. Implement a URL titled /convert which takes in the ID numb
@app.get("/convert", response_model=ConvertResponse)
async def convert(id: int):
    item = await app.db[COLLECTION].find_one({"_id": id})
    if item is None:
        raise HTTPException(status_code=404, detail=f"Product with ID {id} not found")

    async with httpx.AsyncClient() as client:
        try:
            fx_response = await client.get(
                "https://api.frankfurter.dev/v1/latest",
                params={"base": "USD", "symbols": "EUR"},
                timeout=10.0
            )
            fx_response.raise_for_status()
        except httpx.HTTPError:
            raise HTTPException(status_code=503, detail="Could not reach exchange rate API. Try again later.")

    fx_data = fx_response.json()
    eur_rate = fx_data["rates"].get("EUR")
    if eur_rate is None:
        raise HTTPException(status_code=502, detail="Exchange rate data is missing EUR rate")

    rate_date = fx_data["date"]
    price_usd = item["UnitPrice"]
    price_eur = round(price_usd * eur_rate, 2)

    return ConvertResponse(
        ProductID=item["_id"],
        Name=item["Name"],
        price_usd=price_usd,
        price_eur=price_eur,
        exchange_rate=eur_rate,
        rate_date=rate_date
    )
```

Unit test code

GET {{baseUrl}} /getAll

Send

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("Returns array", () => {
    const body = pm.response.json();
    pm.expect(body).to.be.an("array");
});
pm.test("Items have required fields", () => {
    const body = pm.response.json();
    if (body.length > 0) {
        pm.expect(body[0]).to.have.all.keys("ProductID","Name","UnitPrice","StockQuantity",
            "Description");
    }
});
```

GET {{baseUrl}} /getSingleProduct?id= {{testProductId}}

Send

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("Correct product returned", () => {
    const body = pm.response.json();
    pm.expect(body.ProductID).to.eql(pm.collectionVariables.get("testProductId"));
    pm.expect(body.Name).to.be.a("string");
    pm.expect(body.UnitPrice).to.be.a("number");
    pm.expect(body.StockQuantity).to.be.a("number");
    pm.expect(body.Description).to.be.a("string");
});
```

POST {{baseUrl}} /addNew

Docs Params Authorization Headers 8 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 201", () => pm.response.to.have.status(201));
pm.test("Returns created product", () => {
    const body = pm.response.json();
    pm.expect(body.ProductID).to.eql(9999);
    pm.expect(body.Name).to.eql("Test Product");
    pm.expect(body.UnitPrice).to.eql(19.99);
});
```

GET {{baseUrl}} /getSingleProduct?id= {{newId}}

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 200", () => pm.response.to.have.status(200));
```

GET {{baseUrl}} /startsWith?letter=T

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("All names start with T (case-insensitive)", () => {
    const body = pm.response.json();
    body.forEach(item => {
        pm.expect(item.Name[0].toLowerCase()).to.eql("t");
    });
});
```

GET {{baseUrl}} /paginate?start_id=1001&end_id=1050

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("Returns max 10 items", () => {
    const body = pm.response.json();
    pm.expect(body).to.be.an("array");
    pm.expect(body.length).to.be.at.most(10);
});
pm.test("IDs are within range", () => {
    pm.response.json().forEach(item => {
        pm.expect(item.ProductID).to.be.within(1001, 1050);
    });
});
```

GET {{baseUrl}} /convert?id= {{testProductId}} Send

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("Contains EUR conversion fields", () => {
    const body = pm.response.json();
    pm.expect(body).to.have.all.keys("ProductID","Name","price_usd","price_eur","exchange_rate",
        "rate_date");
    pm.expect(body.price_eur).to.be.a("number");
    pm.expect(body.exchange_rate).to.be.a("number");
    pm.expect(body.exchange_rate).to.be.above(0);
});
pm.test("EUR price is calculated correctly", () => {
    const body = pm.response.json();
    const expected = Math.round(body.price_usd * body.exchange_rate * 100) / 100;
    pm.expect(body.price_eur).to.be.closeTo(expected, 0.01);
});
```

DELETE {{baseUrl}} /deleteOne?id= {{newId}}

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 204", () => pm.response.to.have.status(204));
```

GET {{baseUrl}} /getSingleProduct?id= {{newId}}

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```
pm.test("Status 200", () => pm.response.to.have.status(404));
```

Docker config file

```
FROM ubuntu:22.04 (last pushed 1 week ago)

WORKDIR /app

RUN apt-get update && apt-get install -y python3 python3-pip
RUN rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python3", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

API endpoints

GET /getSingleProduct Get Single Product

Parameters

Cancel

Name	Description
id * required integer (query)	1001

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/getSingleProduct?id=1001' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/getSingleProduct?id=1001
```

Server response

Code	Details
200	Response body

```
{
  "ProductID": 1001,
  "Name": "NVIDIA RTX 4090",
  "UnitPrice": 1599.99,
  "StockQuantity": 12,
  "Description": "High-end GPU with 24GB VRAM for 4K gaming."
}
```

Download

Response headers

```
content-length: 141
content-type: application/json
date: Sun,29 Mar 2026 19:20:41 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "ProductID": 0,
  "Name": "string",
  "UnitPrice": 0,
  "StockQuantity": 0,
  "Description": "string"
}
```

422	Validation Error	No links

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

GET /getAll Get All

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/getAll' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/getAll
```

Server response

Code	Details
200	Response body


```
[
  {
    "ProductID": 1001,
    "Name": "NVIDIA RTX 4090",
    "UnitPrice": 1599.99,
    "StockQuantity": 12,
    "Description": "High-end GPU with 24GB VRAM for 4K gaming."
  },
  {
    "ProductID": 1002,
    "Name": "AMD Ryzen 9 7950X",
    "UnitPrice": 549,
    "StockQuantity": 25,
    "Description": "16-core 32-thread enthusiast desktop processor."
  },
  {
    "ProductID": 1003,
    "Name": "Samsung 990 Pro 2TB",
    "UnitPrice": 179.99,
    "StockQuantity": 40,
    "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
  },
  {
    "ProductID": 1004,
    "Name": "Corsair Vengeance 32GB DDR5",
    "UnitPrice": 115,
    "StockQuantity": 60,
    "Description": "6000MHz CL36 Dual Channel Memory Kit."
```


Download

Response headers

```
content-length: 28147
content-type: application/json
date: Sun,29 Mar 2026 19:23:21 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema

```
[
  {
    "ProductID": 0,
    "Name": "string",
    "UnitPrice": 0,
    "StockQuantity": 0,
    "Description": "string"
  }
]
```

/addNew

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/addNew' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "ProductID": 9999,
  "Name": "Test Product",
  "UnitPrice": 19.99,
  "StockQuantity": 50,
  "Description": "A test item"
}'
```

Request URL

```
http://127.0.0.1:8000/addNew
```

Server response

Code	Details
201	Response body

```
{
  "ProductID": 9999,
  "Name": "Test Product",
  "UnitPrice": 19.99,
  "StockQuantity": 50,
  "Description": "A test item"
}
```

Download

Response headers

```
content-length: 105
content-type: application/json
date: Sun,29 Mar 2026 19:24:13 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "ProductID": 0,
  "Name": "string",
  "UnitPrice": 0,
  "StockQuantity": 0,
  "Description": "string"
}
```

422	Validation Error	*No links*

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

DELETE /deleteOne Delete One

Parameters

Cancel

Name	Description
id * required integer (query)	9999

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/deleteOne?id=9999' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/deleteOne?id=9999
```

Server response

Code	Details
204	Response headers

```
content-type: application/json
date: Sun,29 Mar 2026 19:25:24 GMT
server: uvicorn
```

Responses

Code	Description	Links
204	Successful Response	*No links*
422	Validation Error	*No links*

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

/startsWith

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/startsWith?letter=t' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/startsWith?letter=t
```

Server response

Code	Details
200	Response body


```
  {
    "ProductID": 1037,
    "Name": "Thermaltake Toughpower 750W",
    "UnitPrice": 105,
    "StockQuantity": 33,
    "Description": "80+ Gold semi-modular power supply."
  },
  {
    "ProductID": 1050,
    "Name": "Thermal Grizzly Kryonaut",
    "UnitPrice": 11.99,
    "StockQuantity": 150,
    "Description": "High-end thermal grease for overclocking."
  },
  {
    "ProductID": 1059,
    "Name": "Toshiba X300 10TB",
    "UnitPrice": 220,
    "StockQuantity": 10,
    "Description": "High-performance 3.5-inch desktop HDD."
  },
  {
    "ProductID": 1082,
    "Name": "Teamgroup Elite 8GB",
    "UnitPrice": 22,
    "StockQuantity": 150,
    "Description": "Basic 3200MHz DDR4 SODIMM for laptops."
  },
  {
    "ProductID": 1086,
```


Download

Response headers

```
content-length: 1832
content-type: application/json
date: Sun,29 Mar 2026 19:25:56 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema


```
[
  {
    "ProductID": 0,
    "Name": "string",
    "UnitPrice": 0,
    "StockQuantity": 0,
    "Description": "string"
  }
]
```


422	Validation Error	*No links*

Media type

application/json

Example Value | Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

/paginate

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/paginate?start_id=1001&end_id=1050' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/paginate?start_id=1001&end_id=1050
```

Server response

Code	Details
200	Response body


```
[
  {
    "ProductID": 1001,
    "Name": "NVIDIA RTX 4090",
    "UnitPrice": 1599.99,
    "StockQuantity": 12,
    "Description": "High-end GPU with 24GB VRAM for 4K gaming."
  },
  {
    "ProductID": 1002,
    "Name": "AMD Ryzen 9 7950X",
    "UnitPrice": 549,
    "StockQuantity": 25,
    "Description": "16-core 32-thread enthusiast desktop processor."
  },
  {
    "ProductID": 1003,
    "Name": "Samsung 990 Pro 2TB",
    "UnitPrice": 179.99,
    "StockQuantity": 40,
    "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
  },
  {
    "ProductID": 1004,
    "Name": "Corsair Vengeance 32GB DDR5",
    "UnitPrice": 115,
    "StockQuantity": 60,
    "Description": "6000MHz CL36 Dual Channel Memory Kit."
```


Download

Response headers

```
content-length: 1449
content-type: application/json
date: Sun,29 Mar 2026 19:27:11 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema


```
[
  {
    "ProductID": 0,
    "Name": "string",
    "UnitPrice": 0,
    "StockQuantity": 0,
    "Description": "string"
  }
]
```


422	Validation Error	*No links*

Media type

application/json

Example Value | Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

GET /convert Convert

Parameters

Cancel

Name	Description
id * required integer (query)	1001

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/convert?id=1001' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/convert?id=1001
```

Server response

Code	Details
200	Response body

```
{
  "ProductID": 1001,
  "Name": "NVIDIA RTX 4090",
  "price_usd": 1599.99,
  "price_eur": 1389.24,
  "exchange_rate": 0.86828,
  "rate_date": "2026-03-27"
}
```

Download

Response headers

```
content-length: 132
content-type: application/json
date: Sun,29 Mar 2026 19:28:03 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "ProductID": 0,
  "Name": "string",
  "price_usd": 0,
  "price_eur": 0,
  "exchange_rate": 0,
  "rate_date": "string"
}
```

422	Validation Error	No links

Media type

application/json

Example Value | Schema

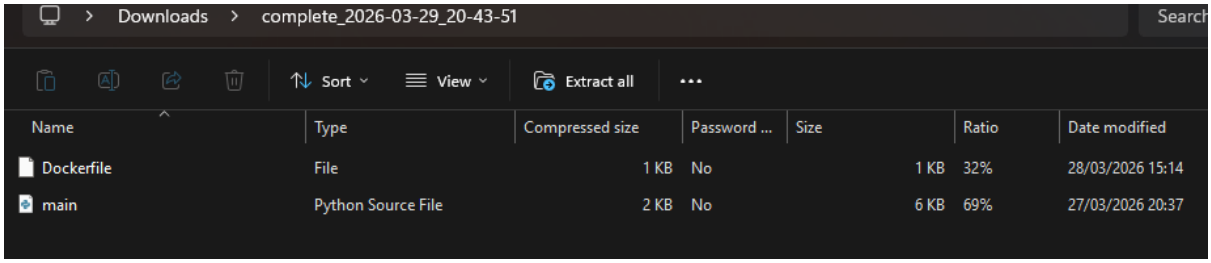
```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

```
C:\Users\davyt\OneDrive - Technological University Dublin\College\Sem8\Web Services\Web-Services-Jenkins-Postman>uvicorn
 main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\davyt\\OneDrive - Technological University Dublin\\C
ollege\\Sem8\\Web Services\\Web-Services-Jenkins-Postman']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [28468] using WatchFiles
INFO:     Started server process [27044]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:58076 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:58076 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:62152 - "GET /getSingleProduct?id=1001 HTTP/1.1" 200 OK
INFO:     127.0.0.1:63985 - "GET /getAll HTTP/1.1" 200 OK
INFO:     127.0.0.1:56265 - "POST /addNew HTTP/1.1" 201 Created
INFO:     127.0.0.1:58120 - "DELETE /deleteOne?id=9999 HTTP/1.1" 204 No Content
INFO:     127.0.0.1:52471 - "GET /startsWith?letter=t HTTP/1.1" 200 OK
INFO:     127.0.0.1:54431 - "GET /paginate?start_id=1001&end_id=1050 HTTP/1.1" 200 OK
INFO:     127.0.0.1:53653 - "GET /convert?id=1001 HTTP/1.1" 200 OK
```

Jenkins

```
pipeline {
    agent any
    stages {
        stage('clone') {
            steps {
                echo 'Get code from github repo'
                bat 'dir'
                checkout scmGit(branches: [[name: 'main']], userRemoteConfigs: [[url: 'https://github.com/DavidThorn03/Web-Services-Jenkins-Postman.git']])
                bat 'dir'
            }
        }
        stage('Install Depenancies'){
            steps {
                echo 'Installing dependancies'
                bat 'pip install -r requirements.txt'
                bat 'npm install -g newman'
            }
        }
        stage('Create Env'){
            steps {
                echo 'Create env file'
                withCredentials([file(credentialsId: 'fastapi_env', variable: 'ENV_FILE')]) {
                    bat 'copy %ENV_FILE% .env'
                }
            }
        }
        stage('Build docker'){
            steps {
                echo 'Set up docker with code from git'
                bat 'docker build -t app .'
            }
        }
        stage('Run Container'){
            steps {
                echo 'Run docker container'
                bat 'docker run -d --name app -p 8000:8000 --env-file .env app'
            }
        }
        stage('Run tests') {
            steps {
                echo 'Now time to try test the code..'
                bat 'npx newman run Products.postman_collection.json --reporters cli,junit --reporter-junit-export results/newman-results.xml'
            }
            post {
                always {
                    junit 'results/newman-results.xml'
                }
            }
        }
        stage('Kill docker'){
            steps {
                echo 'Stop docker from running'
                bat 'docker stop app'
                bat 'docker rm app'
            }
        }
        stage('Create zip file'){
            steps {
                echo 'With api code and docker config from github'
                echo 'first delete old files'
                bat 'del complete_*.zip'
                bat "powershell Compress-Archive -Path main.py, Dockerfile -DestinationPath complete_${env.BUILD_TIMESTAMP}.zip"
            }
            post {
                success {
                    archiveArtifacts artifacts: 'complete_*.zip'
                }
            }
        }
    }
}
```

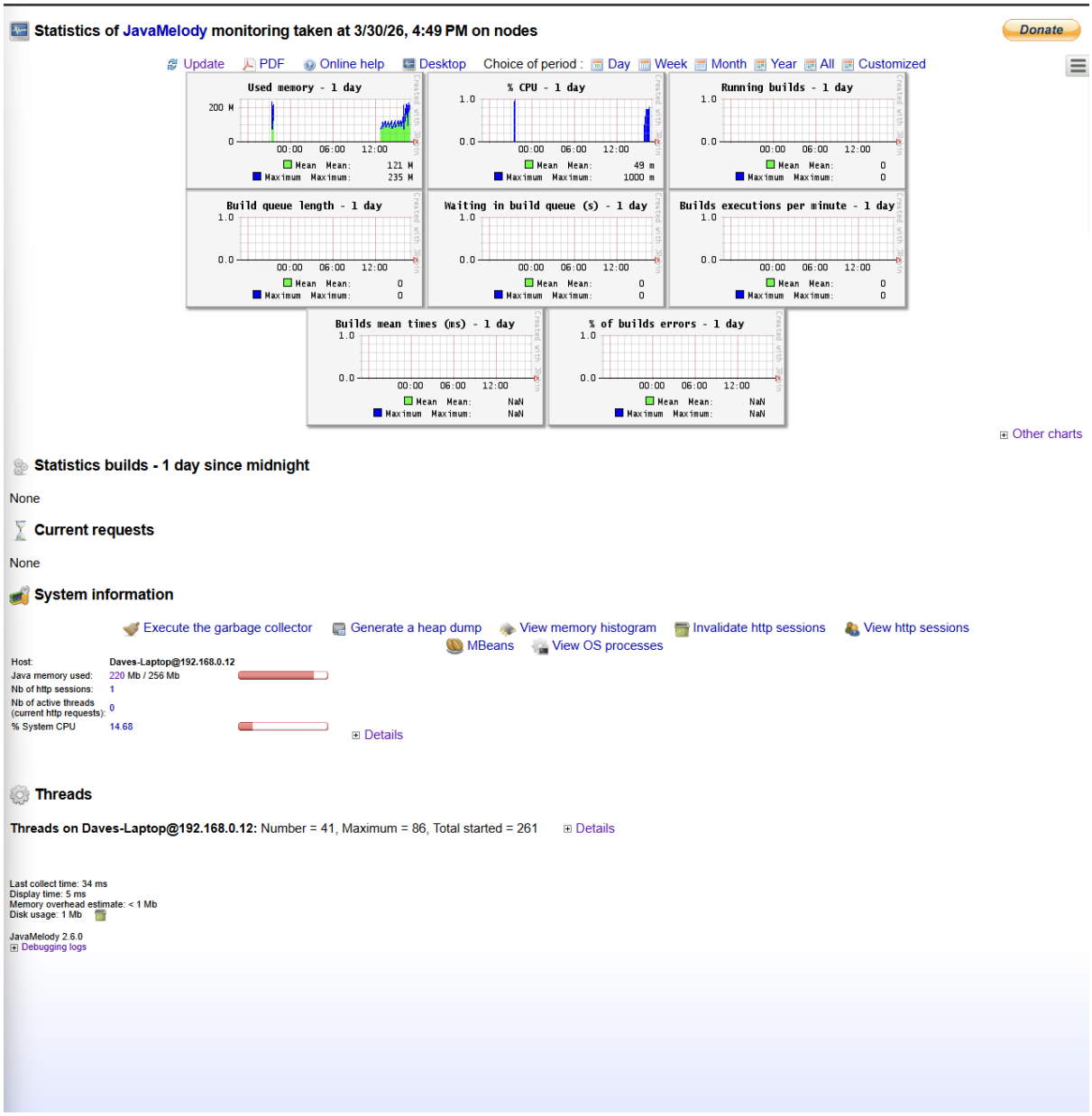
Zip file output



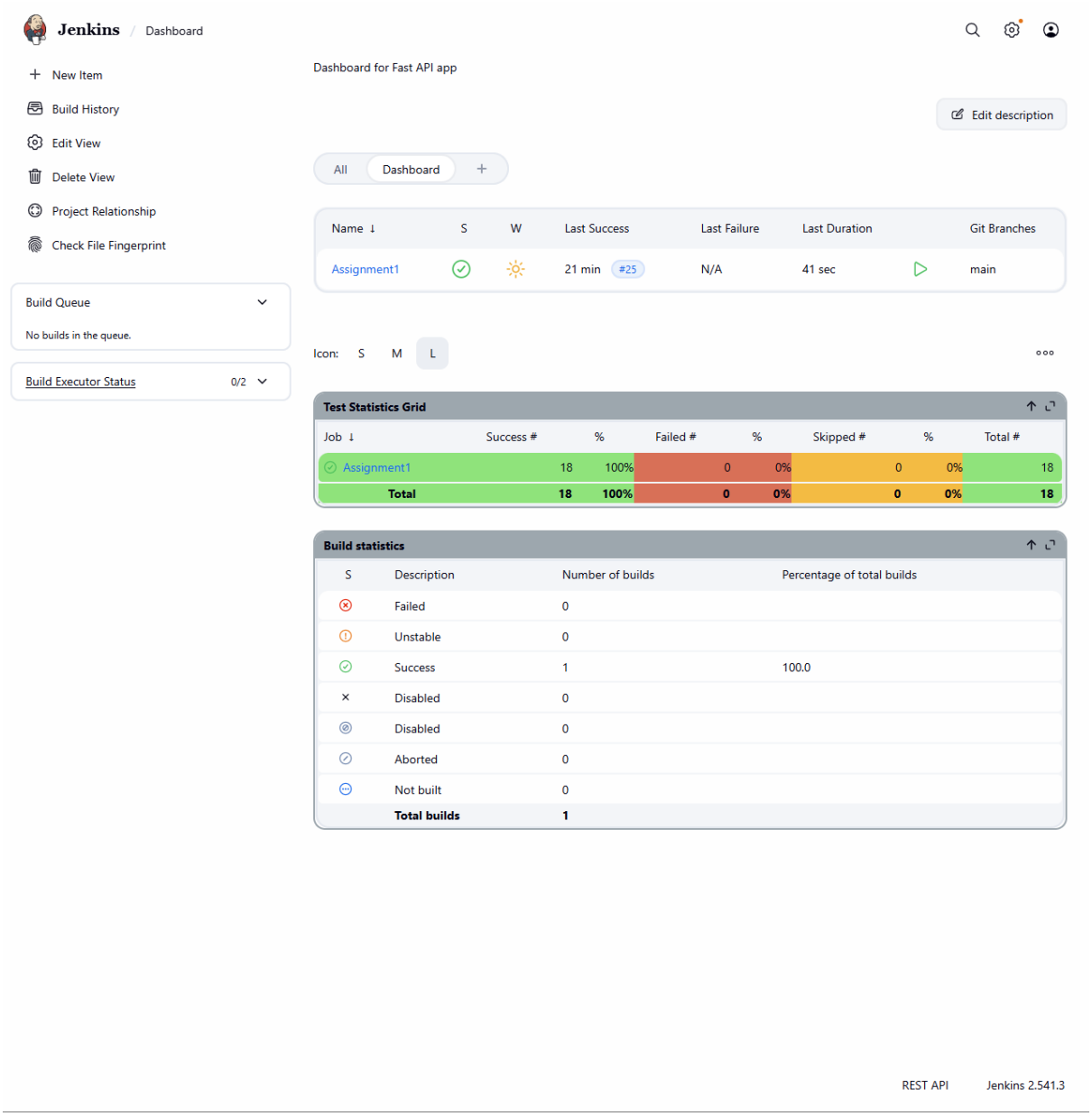
Monitoring tools

For monitoring tools I used several Jenkins plugin, which were used to more easily monitor test and build results, as well as show machine information like memory and cpu usage.

For memory and CPU usage I used the Monitoring plugin, which uses JavaMelody to get information on memory and cpu usage, the number of running build and builds in the queue.



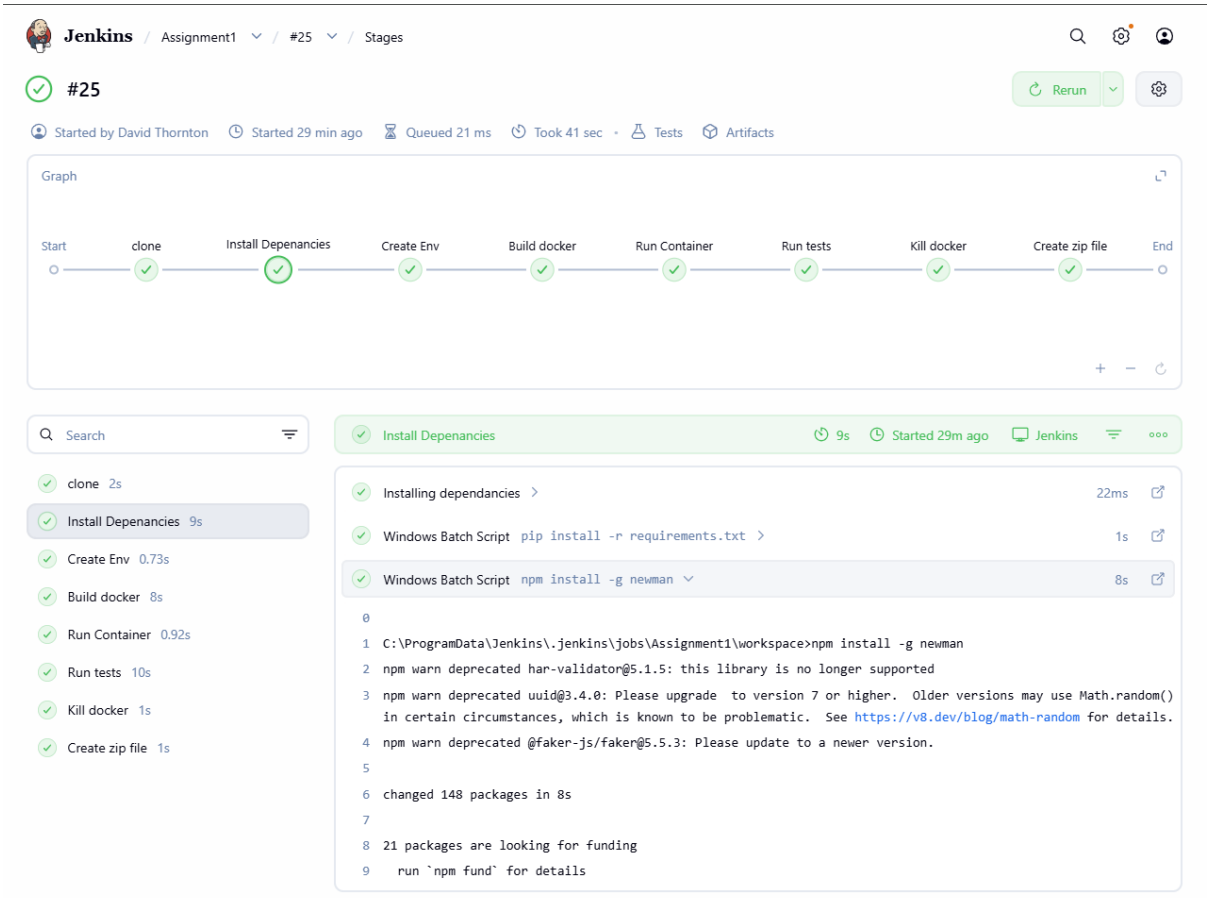
Next, the Dashboard view was used to monitor build statistics and test results. It can also be used to monitor other aspects like job statistics. It can also be used to further customize the main page view to show more information about each of the pipelines, like the git branches it is using.



NOTE: To get the test statistics seen here, new lines need to be added to the test stage of the pipeline.

```
stage('Run tests') {
    steps {
        echo 'Now time to try test the code..'
        bat 'npx newman run Products.postman_collection.json --reporters cli,junit --reporter-junit-export results/newman-results.xml'
    }
    post {
        always {
            junit 'results/newman-results.xml'
        }
    }
}
```

Finally, the job and stage monitoring plugin was used. This allows you to more clearly see how jobs perform on a stage by stage level. You can use this both to compare the results of several previous jobs and job specific results.

Note: There would be more jobs shown here, the pipeline used here only saves the most recent successful job and hence, only one is shown.

This plugin makes it easier to see not just stage by stage, but line by line results.